RISC-V Fields

RISC-V fields are given names to make them easier to discuss:

funct7	rs2	rs1	funct3	rd	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Here is the meaning of each name of the fields in RISC-V instructions:

- *opcode:* Basic operation of the instruction, and this abbreviation is its traditional name.
- *rd:* The register destination operand. It gets the result of the operation.
- *funct3:* An additional opcode field.
- *rs1:* The first register source operand.
- *rs2:* The second register source operand.
- *funct7:* An additional opcode field.

opcode

The field that denotes the operation and format of an instruction.

A problem occurs when an instruction needs longer fields than those shown above. For example, the load register instruction must specify two registers and a constant. If the address were to use one of the 5-bit fields in the format above, the largest constant within the load register instruction would be limited to only 2^5-1 or 31. This constant is used to select elements from arrays or data structures, and it often needs to be much larger than 31. This 5-bit field is too small to be useful.

Hence, we have a conflict between the desire to keep all instructions the same length and the desire to have a single instruction format. This conflict leads us to the final hardware design principle:

Design Principle 3: Good design demands good compromises.

The compromise chosen by the RISC-V designers is to keep all instructions the same length, thereby requiring distinct instruction formats for different kinds of instructions. For example, the format above is called *R-type* (for register). A second type of instruction format is *I-type* and is used by arithmetic operands with one constant operand, including `addi`, and by load instructions. The fields of the I-type format are

immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

The 12-bit immediate is interpreted as a two's complement value, so it can represent integers from -2^{11} to $2^{11}-1$. When the I-type format is used for load instructions, the immediate represents a byte offset, so the load doubleword instruction can refer to any doubleword within a region of $\pm 2^{11}$ or 2048 bytes ($\pm 2^{8}$ or 256 doublewords) of the base address in the base register rd. We see that more than 32 registers would be difficult in this format, as the rd and rs1 fields would each need another bit, making it harder to fit everything in one word.

Let's look at the load register instruction from page 71:

```
ld x9, 64(x22) // Temporary reg x9 gets A[8]
```

Here, 22 (for `x22`) is placed in the rs1 field, 64 is placed in the immediate field, and 9 (for `x9`) is placed in the rd field. We also need a format for the store doubleword instruction, `sd`, which needs two source registers (for the base address and the store data) and an immediate for the address offset. The fields of the S-type format are

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

The 12-bit immediate in the S-type format is split into two fields, which supply the lower 5 bits and upper 7 bits. The RISC-V architects chose this design because it keeps the rs1 and rs2 fields in the same place in all instruction formats. Keeping the instruction formats as similar as possible reduces hardware complexity.

Similarly, the opcode and funct3 fields are the same size in all locations, and they are always in the same place.

In case you were wondering, the formats are distinguished by the values in the opcode field: each format is assigned a distinct set of opcode values in the first field (opcode) so that the hardware knows how to treat the rest of the instruction. Figure 2.5 shows the numbers used in each field for the RISC-V instructions covered so far.

Translating RISC-V Assembly Language into Machine Language

Example

We can now take an example all the way from what the programmer writes to what the computer executes. If `x10` has the base of the array `A` and `x21` corresponds to `h`, the assignment statement

```
A[30] = h + A[30] + 1;
```

is compiled into

```
ld x9, 240(x10) // Temporary reg x9 gets A[30]
add x9, x21, x9 // Temporary reg x9 gets h+A[30]
addi x9, x9, 1 // Temporary reg x9 gets h+A[30]+1
sd x9, 240(x10) // Stores h+A[30]+1 back into A[30]
```

What is the RISC-V machine language code for these three instructions?

Answer

For convenience, let's first represent the machine language instructions using decimal numbers. From Figure 2.5, we can determine the three machine language instructions:

immediate	rs1	funct3	rd	opcode
240	10	3	9	3

funct7	rs2	rs1	funct3	rd	opcode
0	9	21	0	9	51

immediate	rs1	funct3	rd	opcode
1	9	0	9	19

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
7	9	10	3	16	35

The `ld` instruction is identified by 3 (see Figure 2.5) in the opcode field and 3 in the funct3 field. The base register 10 is specified in the rs1 field, and the destination register 9 is specified in the rd field. The offset to select `A[30]` (240=30×8) is found in the immediate field.

The `add` instruction that follows is specified with 51 in the opcode field, 0 in the funct3 field, and 0 in the funct7 field. The three register operands (9, 21, and 9) are found in the rd, rs1, and rs2 fields.

The subsequent `addi` instruction is specified with 19 in the opcode field and 0 in the funct3 field. The register operands (9 and 9) are found in the rd and rs1 fields, and the constant addend 1 is found in the immediate field.

The `sd` instruction is identified with 35 in the opcode field and 3 in the funct3 field. The register operands (9 and 10) are found in the rs2 and rs1 fields, respectively. The address offset 240 is split across the two immediate fields. Since the upper part of the immediate holds bits 5 and above, we can decompose the offset 240 by dividing by 2^5. The upper part of the immediate holds the

quotient, 7, and the lower part holds the remainder, 16.

Since 240_{ten} = $0000\ 1111\ 0000_{two}$, the binary equivalent to the decimal form is:

immediate	rs1	funct3	rd	opcode
000011110000	01010	011	01001	0000011

funct7	rs2	rs1	funct3	rd	opcode
0000000	01001	10101	000	01001	0110011

immediate	rs1	funct3	rd	opcode
000000000001	01001	000	01001	0010011

immediate[11:5]	rs2	rs1	funct3	immediate[4:0]	opcode
0000111	01001	01010	011	10000	0100011

Elaboration

RISC-V assembly language programmers aren't forced to use `addi` when working with constants. The programmer simply writes `add`, and the assembler generates the proper opcode and the proper instruction format depending on whether the operands are all registers (R-type) or if one is a constant (I-type). We use the explicit names in RISC-V for the different opcodes and formats as we think it is less confusing when introducing assembly language versus machine language.

Elaboration

Although RISC-V has both `add` and `sub` instructions, it does not have a `subi` counterpart to `addi`. This is because the immediate field represents a two's complement integer, so `addi` can be used to subtract constants.

Hardware/Software Interface

The desire to keep all instructions the same size conflicts with the desire to have as many registers as possible. Any increase in the number of registers uses up at least one more bit in every register field of the instruction format. Given these constraints and the design principle that smaller is faster, most instruction sets today have 16 or 32 general-purpose registers.

Instruction	Format	funct7	rs2	rs1	funct3	rd	opcode
add (add)	R	0000000	reg	reg	000	reg	0110011
sub (sub)	R	0100000	reg	reg	000	reg	0110011
Instruction	**Format**	**immediate**		**rs1**	**funct3**	**rd**	**opcode**
addi (add immediate)	I	constant		reg	000	reg	0010011
ld (load doubleword)	I	address		reg	011	reg	0000011
Instruction	**Format**	**immed -iate**	**rs2**	**rs1**	**funct3**	**immed -iate**	**opcode**
sd (store doubleword)	S	address	reg	reg	011	address	0100011

FIGURE 2.5 RISC-V instruction encoding.
In the table above, "reg" means a register number between 0 and 31 and "address" means a 12-bit address or constant. The funct3 and funct7 fields act as additional opcode fields.

Figure 2.6 summarizes the portions of RISC-V machine language described in this section. As we shall see in Chapter 4, the similarity of the binary representations of related instructions simplifies hardware design. These similarities are another example of regularity in the RISC-V architecture.

R-type Instructions	funct7	rs2	rs1	funct3	rd	opcode	Example
add (add)	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sub)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3
I-type Instructions	**immediate**		**rs1**	**funct3**	**rd**	**opcode**	**Example**
addi (add immediate)	001111101000		00010	000	00001	0010011	addi x1, x2, 1000
ld (load doubleword)	001111101000		00010	011	00001	0000011	ld x1, 1000 (x2)
S-type Instructions	**immed -iate**	**rs2**	**rs1**	**funct3**	**immed -iate**	**opcode**	**Example**
sd (store doubleword)	0011111	00001	00010	011	01000	0100011	sd x1, 1000(x2)

FIGURE 2.6 RISC-V architecture revealed through Section 2.5.
The three RISC-V instruction formats so far are R, I, and S. The R-type format has two source register operand and one destination register operand. The I-type format replaces one source register operand with a 12-bit *immediate* field. The S-type format has two source operands and a 12-bit immediate field, but no destination register operand. The S-type immediate field is split into two parts, with bits 11—5 in the leftmost field and bits 4—0 in the second-rightmost field.

The BIG Picture

Today's computers are built on two key principles:

1. Instructions are represented as numbers.
2. Programs are stored in memory to be read or written, just like data.

These principles lead to the *stored-program* concept; its invention let the computing genie out of its bottle. Figure 2.7 shows the power of the concept; specifically, memory can contain the source code for an editor program, the corresponding compiled machine code, the text that the compiled program is using, and even the compiler that generated the machine code.